Azure SQL Elastic Pool Operations Runbook

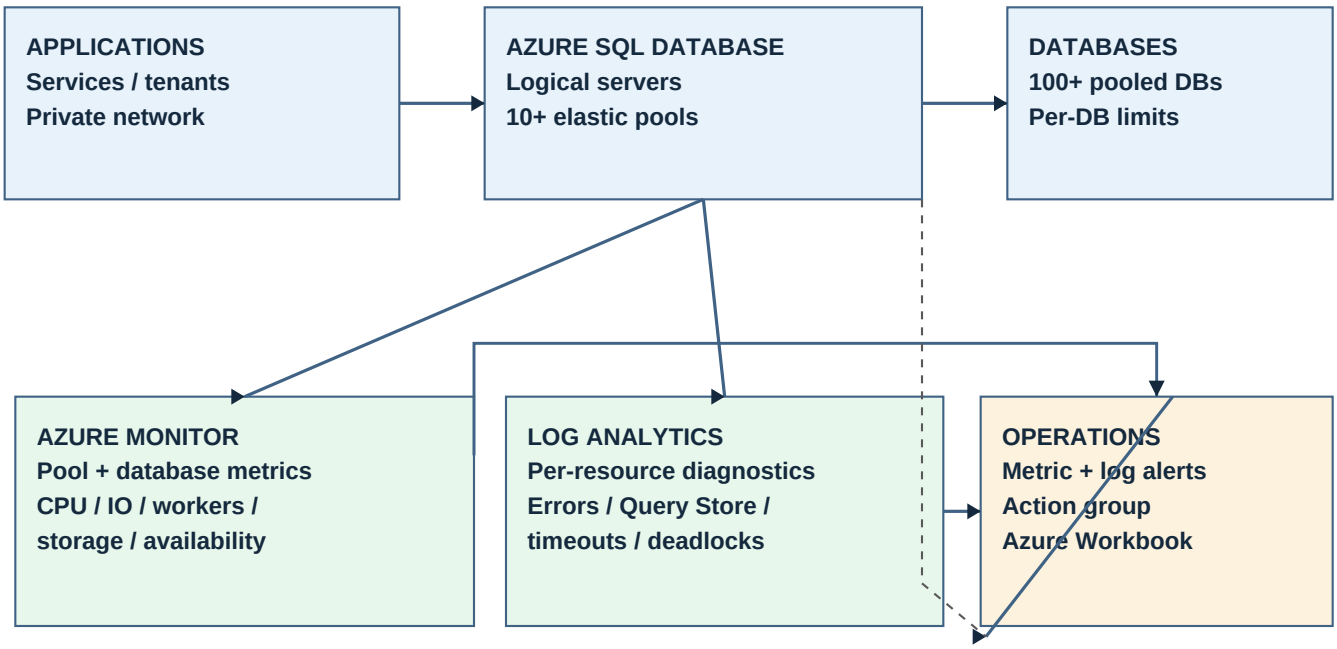
Architecture, provisioning, observability, alerting, daily support, and regional recovery

Reference scale: 10+ elastic pools / 100+ databases

Azure SQL Database - operational reference; validate values for your environment.

Project: azure-sql-elastic-pool-operations-runbook

REFERENCE ARCHITECTURE



Applications connect to workload-aligned Azure SQL pools. Azure Monitor collects platform metrics; diagnostic settings export pool and database telemetry to Log Analytics. Alert rules route actionable signals to the on-call team, and a Workbook provides fleet-to-database drilldown. A separately tested recovery region supports the approved continuity plan.

Diagram is a reference layout, not a deployed environment. Confirm network topology, regional redundancy, product support, SKU, cost, and RPO/RTO with the service owners.

Azure SQL Elastic Pool Operations Runbook

Purpose: Build and operate a shared Azure SQL Database estate with 10+ elastic pools and 100+ databases while meeting agreed availability, performance, security, and cost objectives.

Audience: Azure SQL DBAs, platform/SRE teams, application owners, and incident commanders.

Service scope: Azure SQL Database elastic pools. Azure SQL Managed Instance, SQL Server on VMs, and Azure Synapse have different operational surfaces and are out of scope.

Document status: Reference implementation and operational starting point. Replace sample names and thresholds with approved environment values.

> **Before production: Approve the service inventory, application owners, service-level indicators/objectives, alert routing, change windows, retention, security policy, and recovery point/time objectives. This guide does not provide a universal capacity size or guarantee an SLA.**

1. Goals and operating model

Target outcomes

- Centrally manage pool/database ownership, capacity, costs, and lifecycle.
- Detect saturation at the pool and individual database level before it breaches application objectives.
- Route actionable alerts to a staffed on-call path and retain diagnostic evidence for investigation.
- Automate repeatable provisioning and safe changes through reviewed infrastructure as code.
- Recover from regional or logical incidents using tested restore and failover procedures.

Fleet assumptions

The examples assume at least 10 pools and 100 pooled databases. Use a resource inventory and workload classes rather than relying on a fixed database-per-pool ratio. Azure SQL elastic pools charge for provisioned pool resources, not the number of databases or their momentary consumption; validate cost and SKU behavior against the current pricing and service documentation.

Service records to maintain

For each database, record at minimum:

Field	Example / rule
Environment, application, tenant, owner	Must be populated and kept current.
Business criticality and on-call group	Identifies escalation route.
Logical server, region, pool, SKU/model	Source of truth for placement and capacity.
Database max/min resources and storage limit	Record actual configured bounds.
Application SLO/SLA, RPO, RTO	Obtain from service owner; do not invent.
Maintenance window and dependency	Include application release/peak windows.
Recovery topology and last successful test	Include failover-group membership or restore approach.
Cost center, data classification, retention	Drives budget, security, and retention controls.

Use tags consistently at supported Azure resource scopes, such as environment, application, owner, costCenter, criticality, and dataClassification. Do not put secrets or personal data in tags.

2. Architecture

The component diagram and glossary are in architecture.md; the source diagram is in ../diagrams/azure-sql-elastic-pool-platform.mmd.
In brief: applications connect over approved network paths to logical servers; workload-aligned pools share capacity among bounded databases; platform metrics and per-resource diagnostic settings feed Azure Monitor and Log Analytics; alert rules notify the action group; an Azure Monitor Workbook provides fleet and drill-down views; and a separately tested recovery region supports the defined business continuity plan.

Design decisions

1. Workload isolation: Place databases with compatible workload, latency, tenant, data-residency, and criticality requirements together. Keep sustained high-utilization and unpredictable critical workloads isolated when noisy-neighbor risk outweighs sharing savings.

2. Sizing: Observe representative peak and seasonal windows. Evaluate pool CPU, data IO, log IO, workers, storage, plus each hot database's contribution and workload/query trends. Allow headroom for correlated bursts, deployment/migration events, and failover.

3. Database bounds: Set per-database minimum/maximum resource limits based on application demand and fairness. Verify the limits and pool SKU support the chosen configuration. A maximum can prevent one database from consuming the pool but can also throttle that database.

4. Tier/model: Select DTU or vCore and service tier/features using current requirements, feature support, performance evidence, and cost analysis. DTU dtu_consumption_percent and vCore cpu_percent are not interchangeable.

5. Regional recovery: Decide recovery approach per service. Failover groups use stable read-write/read-only listener endpoints, but the application, DNS, identity, network path, dependencies, and secondary capacity must all be tested. A forced failover can lose data; only the service owner/incident commander can authorize it under the recovery policy.

3. Azure components and definitions

Component | Operational definition
Resource group and subscription | Azure management, RBAC, policy, and billing boundaries. Separate environments and align grouping with ownership/operations.
SQL logical server | Azure SQL Database management/connection boundary. It is a platform resource, not a customer-managed VM.
Elastic pool | Shared DTU- or vCore-based capacity budget containing Azure SQL databases on a logical server. The pool can be scaled; databases can have individual bounds.
Pooled database | A database assigned to one elastic pool. Pool metrics show aggregate pressure; database metrics and query telemetry help identify which tenant/workload causes it.
Private Endpoint / Private DNS | Private connectivity and name resolution for the logical server. Validate DNS from application subnets and ensure public network access follows policy.
Azure Monitor metrics | Near-real-time platform metrics for resource consumption and health. Metric availability varies by resource type and SKU.
Diagnostic setting | Resource-scoped export of metrics and logs to Log Analytics, Storage, and/or Event Hubs. Configure pool and database independently.
Log Analytics workspace | Central retention/query location for exported monitoring data. Control RBAC, network access, retention, and ingestion cost.
Metric alert / scheduled-query alert | Azure Monitor rules for numeric platform metrics or Log Analytics query results, respectively.
Action group | Reusable email/SMS/voice/ITSM/webhook/automation destination. Use owned, tested, secure, and staffed receivers.
Azure Monitor Workbook | Interactive, parameterized dashboard built from Azure Monitor metrics/logs and explanatory text.
Bicep / Terraform / CI | Versioned, reviewed deployment and policy mechanism. Prefer short-lived workload federation and least-privilege identities over stored credentials.
SQL auditing / Defender for SQL | Security auditing and threat-protection capabilities; configure to the organization's compliance and response requirements.
Failover group | Geo-replication/failover abstraction for a logical server's selected databases with stable listener endpoints. It does not replace an application-level DR plan or restore testing.

4. Build and onboard step by step

Step 1 - Discover and classify the fleet

1. Select the subscription(s) and regions. Confirm Azure SQL quotas, allowed SKUs, regional availability, naming/tagging policy, and cost budget.

2. Inventory every logical server, pool, database, SKU, region, network exposure, diagnostics configuration, and failover/backup configuration.

3. Ask each application owner for usage peaks, connection behavior, planned batch jobs, maintenance window, growth estimate, dependencies, SLO, RPO, RTO, and escalation contacts.

4. Group databases by compatible burst shape, criticality, compliance, region, and tenant/workload isolation. Flag databases with persistent high usage for a dedicated tier/pool review.

5. Record current measured pool and database utilization across a representative

business and batch cycle before right-sizing.

Step 2 - Establish management, identity, and network guardrails

1. Separate production and nonproduction subscriptions/resource groups or management boundaries according to the enterprise landing zone.
2. Assign least-privilege Azure RBAC groups for platform operators, DBAs, application owners, security, and read-only responders. Use PIM/JIT for elevated roles; separate deployment identities from human identities.
3. Configure Microsoft Entra administration/authentication, auditing, and security policy. Store application secrets only in an approved secret manager (prefer managed identity where supported).
4. Create private endpoints and private DNS links as required. Test the SQL server hostname resolves to the private endpoint from application networks; test routing and TLS/authentication. Disable public access when private-only is the approved design.
5. Apply Azure Policy for allowed locations/SKUs, diagnostic settings, tags, private access, and security baselines where supported. Validate policy assignments do not block required operations.

Step 3 - Deploy central monitoring resources

The project includes
../infra/monitoring/main.bicep to create a
Log Analytics workspace and email action group. Choose the approved region, retention, resource names, and staffed mailbox. Create a dedicated resource group, then from the repository root run:

```
az login
az account set --subscription "<subscription-id>"
az group create --name "rg-sql-monitoring-prod" --location "eastus"
az deployment group create \
--resource-group "rg-sql-monitoring-prod" \
--template-file "infra/monitoring/main.bicep" \
--parameters location="eastus" \
workspaceName="law-sql-prod" \
actionGroupName="ag-sql-oncall-prod" \
operatorEmail="dba-oncall@example.com" \
retentionInDays=90
```

Examples such as eastus, names, email, and 90 days are placeholders; select values that comply with availability, unique-name, cost, access, and retention requirements. Verify deployment output and use the resulting workspace and action group resource IDs. Add secondary action receivers (ITSM/on-call platform) through approved processes and test notifications.

Step 4 - Provision logical servers, pools, and databases

Use Bicep/Terraform modules and a protected CI workflow for repeatable provisioning. For an existing environment, use a controlled migration plan, not bulk moves during a peak period.

For each logical server:

1. Configure approved region, identity/authentication, network, auditing, Defender/security settings, and tags.
2. Create pools sized from the measured workload class. Validate edition, compute model, hardware family, capacity range, storage, region, and quotas using az sql elastic-pool list-editions and the Azure SQL pricing/limits documentation. SKU options can vary by region and subscription.
3. Create databases into the intended pool and set per-database resource bounds where appropriate. Keep an authoritative mapping of application/tenant -> database -> pool -> owner.
4. Validate connectivity and representative load from the application network; verify application retry/backoff behavior and connection pooling.

Example illustrative vCore pool CLI flow (select supported values for your

region and workload; this is not a production sizing recommendation):

```
az sql elastic-pool list-editions \
--location "eastus" \
--edition GeneralPurpose \
--show-details
az sql elastic-pool create \
--resource-group "rg-sql-prod" \
--server "sql-prod-east-01" \
--name "ep-customer-standard-01" \
--edition GeneralPurpose \
--family Gen5 \
--capacity 4 \
--compute-model Provisioned
az sql db create \
--resource-group "rg-sql-prod" \
--server "sql-prod-east-01" \
--name "customer-001" \
--elastic-pool "ep-customer-standard-01"
```

Check the current CLI help and service documentation before using commands: CLI flags and supported combinations can change. Never apply example capacity to production without workload analysis and cost approval.

Step 5 - Configure diagnostic settings at both scopes

Azure SQL does not export all diagnostics automatically. Configure the elastic-pool resource and each database resource separately:

1. In Azure portal, open the elastic pool resource -> Monitoring -> Diagnostic settings -> Add diagnostic setting.
2. Select the approved Log Analytics workspace. Enable the pool's Basic metric category where offered; route to Storage/Event Hubs additionally only when required.
3. Repeat for every database. Enable Basic metrics and relevant resource log categories supported by the resource. Common database log selections include SQLInsights, AutomaticTuning, QueryStoreRuntimeStatistics, QueryStoreWaitStatistics, Errors, DatabaseWaitStatistics, Timeouts, Blocks, and Deadlocks. Confirm categories with the resource's Diagnostic settings UI/API; not every category or metric applies to every SKU.
4. Choose resource-specific tables when supported and desired; otherwise document the AzureDiagnostics schema in use. Confirm metric and log data arrive in the workspace, and set ingestion/retention budgets.
5. At 10+ pools / 100+ databases, deploy settings via policy/IaC or an idempotent automation process. Reconcile the resource inventory regularly so newly created databases do not silently lack diagnostics.

The Azure SQL documentation explicitly distinguishes pool telemetry from each database's telemetry. A pool diagnostic setting does not cover its member databases.

Step 6 - Create actionable alerting

Create an action group first and test that an alert reaches the correct staffed team. In Azure portal, open Monitor -> Alerts -> Create -> Alert rule. Scope each metric alert to the correct pool/database resource or a supported resource group; select metric, aggregation, threshold, evaluation frequency, lookback window, severity, and action group. Use alert dimensions/splitting when the selected metric offers them. Create separate metric alerts for different signals; avoid combining unrelated thresholds into one condition.

Starting thresholds-not universal SLAs:

Signal | Scope | Starting condition | Evaluation/window | Initial severity / response
CPU percentage (cpu_percent, vCore) or DTU consumption (dtu_consumption_percent, DTU) | Pool; optionally hot DB | Average > 90% | 1 minute / 10 minutes | Sev 2 warning; correlate database contributors, query changes, and load before scaling.
Data IO (physical_data_read_percent) | Pool | Average > 90% | 1 minute / 15 minutes | Sev 3; inspect IO-bound query/workload and storage/read pattern.

Log IO (log_write_percent) | Pool | Average > 90% | 1 minute / 10 minutes | Sev 2; inspect write bursts, transactions, index maintenance, and throttling.
Workers (workers_percent) | Pool/database when available | Metric documentation recommends a minimum > 60% example; treat as early warning and tune | 1 minute / 5 minutes | Sev 2; inspect connection/request concurrency and blocking.
Storage (storage_percent / applicable pool storage metric) | Pool and DB | > 85% sustained; add a higher critical threshold before exhaustion | 5 minutes / 15 minutes | Sev 2 warning; plan growth/cleanup and verify configured max size.
Availability (availability) | Database | Below the service's agreed availability target; brief 0% should trigger incident triage | 1 minute / short approved window | Sev 1/2 per business criticality; correlate Azure Service Health and app synthetic checks.
Deadlocks (deadlock) | Database | Any meaningful increase from baseline; alert on sustained or critical DB-specific rate | 5 minutes / 15 minutes | Sev 3/2; inspect deadlock graphs, transactions, query plan, and application retries.
System failed connections (connection_failed) | Database | Baseline-relative increase; split by error/driver dimension where supported | 1 minute / 5 minutes | Sev 2; distinguish transient service errors from firewall/authentication/user errors.
Query timeouts/errors/blocks | Database logs | Scheduled query count/rate exceeds the per-app baseline | 5 minutes / 15 minutes | Sev 2/3; query the relevant resource-specific table or AzureDiagnostics.
Resource missing diagnostics | Fleet inventory/reconciliation | Any production pool/database missing required setting | Daily reconciliation | Sev 3; remediate as a configuration/compliance gap.

Threshold guidance in Azure SQL metric documentation is a starting point. Baseline each workload and adjust alert duration, severity, and routing to prevent noise. CPU alone is not a health/SLA signal: inspect data/log IO, workers, storage, availability, connections, application latency, and errors together. Some signals are supported only on a particular purchasing model. Use the current metric reference for each pool/database.

For log alerts, create a scheduled-query rule against the workspace using the actual schema selected for diagnostic export. Example for legacy AzureDiagnostics mode (adapt column/category/table names to the workspace):

```
AzureDiagnostics
| where TimeGenerated > ago(15m)
| where ResourceProvider =~ "MICROSOFT.SQL"
| where Category in ("Errors", "Timeouts", "Blocks", "Deadlocks")
| summarize Events = count() by ResourceId, Category, bin(TimeGenerated, 5m)
| where Events > 0
```

Validate the query returns data before making it an alert. Resource-specific tables may have different names and schemas. Use a separate query per signal when different severities or thresholds are needed. Do not page on every SQL error without filtering expected application/user errors and establishing a baseline.

Step 7 - Build the fleet Workbook dashboard

1. In the Azure portal, open Monitor -> Workbooks -> New. Select the Log Analytics workspace as the data source for log-based views; add Azure Monitor Metrics queries for platform metrics if you prefer native metric charts.
2. Add a workbook parameter for subscription/resource group, environment, logical server, and pool. Let responders filter and drill from fleet to pool to database. Use resource selectors rather than a hard-coded list of 100+ database names.
3. Add a Fleet summary row: pool/database counts, active alerts by severity, missing diagnostics, and a table of production pools sorted by highest current utilization.
4. Add synchronized time-series panels for pool CPU/DTU, data IO, log IO, workers, storage, and sessions. Show average and maximum; use the same time range and label each unit/model clearly.
5. Add a database drill-down table: server, pool, database, CPU/DTU, IO, workers, storage, deadlocks/errors where available, owner, environment, and criticality. Include resource links and alert context.
6. Add operational panels: active/fired alerts, SQL diagnostic error/timeout/

blocking trends, service health/change annotations if available, and diagnostics coverage.

7. Add explanatory text with metric semantics, DTU/vCore caveats, on-call instructions, freshness, data-source limitations, and links to the runbook.
8. Save as a shared workbook in the correct subscription/resource group, grant least-privilege access, and validate it with representative production and nonproduction resources. Test the dashboard during an incident simulation.

Example Log Analytics query for a fleet metric trend when Basic metrics are exported to the AzureMetrics table:

```
AzureMetrics
| where TimeGenerated > ago(24h)
| where ResourceProvider =~ "MICROSOFT.SQL"
| where ResourceId has "/elasticPools/"
| where MetricName in (
"cpu_percent",
"dtu_consumption_percent",
"physical_data_read_percent",
"log_write_percent",
"workers_percent",
"storage_percent"
)
| summarize Avg = avg(Average), Peak = max(Maximum)
by Pool = Resource, MetricName, bin(TimeGenerated, 5m)
| order by TimeGenerated asc
```

Metrics and exported table shape depend on SKU and diagnostic destination. Check the workspace schema and use actual metric names returned by Azure Monitor before saving the query. For an at-a-glance current fleet comparison, use Azure Monitor Metrics charts scoped to the pools; do not treat a time-series average as a capacity plan.

To share reliably, save the Workbook as an Azure resource, assign Reader/Workbook Reader to the support group, and document the owning team. Workbooks are not the same as the classic Azure portal dashboard; choose one approved visualization artifact and manage/export it as code where possible.

Step 8 - Automate, secure, and validate

1. Store infrastructure, diagnostic settings, alert definitions, tags, and workbook JSON/export in version control. Require peer review, lint/what-if, a nonproduction deployment, and protected production approval.
2. Use workload identity federation or managed identity where available, least-privilege RBAC, protected secrets, and branch/environment controls.
3. Ensure every new SQL pool/database onboarding pipeline applies diagnostics, tags, alert coverage, ownership, security controls, backup/recovery requirements, and Workbook discoverability.
4. Test alerts using a safe nonproduction rule/test signal and verify the action group. Do not induce production saturation to test paging.
5. Run a read/connection smoke test from the app network. Validate alert data, dashboard filters, diagnostics ingestion delay, and action ownership.
6. Record the baseline and a rollback plan before each scale or placement change; monitor after deployment and reconcile desired state against Azure.

5. Daily operations and common support tasks

Daily on-call checklist

At the start of each support shift and after major deployments:

1. Review active Azure Monitor alerts, incident queue, Azure Service Health, planned maintenance, and changes in the prior shift.
2. Open the SQL fleet Workbook. Review pool CPU/DTU, IO, log IO, workers, storage, availability, and the highest-usage databases for the last 1h, 24h, and 7d.
3. Verify production resource inventory and diagnostics coverage; check

monitoring ingestion freshness and action-group delivery health.

4. Review failed connections, timeouts, deadlocks, blocks, and Query Store regressions in the context of application deployments and workload peaks.

5. Confirm open alerts have an owner, severity, incident/ticket, next update, and mitigation/rollback. Escalate to the application owner when workload semantics or query changes are required.

6. Record significant changes, capacity trends, customer impact, and follow-up work in the operations log.

Common recurring tasks

Cadence | Support task
Daily | Triage alerts/incidents; scan utilization/availability; verify backup/failover status and monitoring freshness; review workload/deployment changes.
Weekly | Review pool peak/percentile trends, hot databases, alert noise, storage growth, query regressions, resource-limit events, and unassigned ownership. Create capacity/cost actions.
Monthly | Report cost by environment/application/pool; compare provisioned capacity to peak demand and agreed headroom; identify idle/underfilled pools, sustained hot pools, unused databases, and retention costs. Review RBAC and diagnostic coverage.
Quarterly | Revalidate SLO/RPO/RTO and escalation roster; test representative restore and regional recovery; review quota, region/SKU availability, security posture, action groups, and runbook accuracy.
On every change | Ticket/approval, impact and cost estimate, pre-change baseline, maintenance window, validation plan, rollback criteria, and post-change observation.

Pool saturation / resource throttling

1. Acknowledge the alert; capture time range, pool metrics, affected applications/databases, recent changes, and error/latency impact.

2. Determine the constrained resource: user CPU/DTU, SQL instance CPU, data IO, log IO, workers/sessions, or storage. Check whether the signal is pool-wide or isolated to one database and compare application demand.

3. Inspect Query Store runtime/wait statistics, top resource-consuming queries, waits, blocking, deadlocks, connection churn, batch jobs, and recent deployments. Coordinate with the application/DB team.

4. Mitigate safely: cancel/pace an approved runaway job, optimize/revert a release/query, correct transaction/connection behavior, isolate a noisy database, or scale the pool within approved SKU/cost constraints.

5. Before scaling, verify capacity, region/SKU/quotas, expected cost, change approval, application impact, and a rollback path. Scaling can trigger transient connection interruptions; coordinate and observe clients.

6. Monitor each resource dimension and application SLO after mitigation. Document root cause, permanent fix, capacity lesson, and alert tuning.

Storage approaching the limit

1. Identify whether pool or a specific database is nearing its configured limit; compare used vs allocated/max size and growth rate.

2. Check unexpected data/log growth, retention/archival, index maintenance, and application changes. Do not delete data without owner approval and retention/legal review.

3. Increase storage/pool limits only after checking SKU limits, cost, change approval, and recovery/backup implications. Confirm space growth stops or is covered, and set an early-warning threshold with runway.

Availability or connection-failure incident

1. Compare Azure SQL Availability/system failed-connection metrics with application synthetic checks, DNS/private endpoint resolution, routing, TLS, Entra/token status, firewall, and recent configuration changes.

2. Check Azure Service Health and SQL activity/operation status. Separate user errors (credentials, firewall, client timeout) from transient system errors; collect UTC timestamps and correlation IDs.

3. Verify connection pool exhaustion, retry/backoff storms, connection strings/listeners, and whether only one tenant/database or a whole server

is affected.

4. Escalate according to the incident matrix. Do not force failover merely because one metric is red; evaluate scope, data-loss risk, replica synchronization, application dependencies, and incident-command approval.

Query regression, blocking, deadlocks, and timeouts

1. Identify affected database, app release, query fingerprint, time window, error code, and concurrency pattern.
2. Review Query Store plans/runtime/waits, blocks/deadlock reports, transaction duration, and index/statistics changes. Compare against the known-good baseline.
3. Coordinate any plan forcing, index change, query rewrite, or job schedule change with the database and application owners; test and document rollback.
4. Validate application-level retries are bounded and idempotent. Retune alerts if only after cause is understood and service objectives are met.

Safe scale-down and cost optimization

1. Require sustained low peak/percentile usage over representative weekdays, batch windows, month-end/seasonal peaks, and release events.
2. Confirm no storage, workers, IO, latency, or per-database-bound constraint will become the limiting factor after scale-down.
3. Obtain owner approval and cost comparison; schedule a reversible change away from peak, observe application SLOs, and document rollback thresholds.
4. Do not optimize for low average CPU alone or pack unrelated tenants solely to reduce pool count.

6. Change, scaling, provisioning, and failover controls

Standard change record

Every production change should state the target resource IDs and owner, business reason, before/after SKU and cost estimate, capacity evidence, expected connection impact, dependency approvals, maintenance window, validation steps, rollback trigger, communications, and observation period. Use peer review and approved CI; avoid ad hoc portal edits that cause drift.

Scaling procedure

1. Identify the sustained bottleneck and affected applications; use metrics and query evidence.
2. Determine whether query/workload remediation, database isolation, or resource scaling is the correct response.
3. Check supported edition/capacity/family, region, quota, constraints, expected bill, and scaling behavior in current Azure documentation.
4. Obtain approval; apply via reviewed IaC/CLI in the maintenance window.
5. Validate pool/database status, app connectivity, latency/error rates, all resource dimensions, and cost. Roll back when agreed criteria are crossed.

Provisioning a database into a pool

Confirm tenant/application owner, data region/classification, pool compatibility and headroom, database max/min limits, network/authentication, tagging, auditing, diagnostics, backups/recovery, alert coverage, and application connection configuration. Provision through CI; validate connection and instrumentation; add to inventory and Workbook coverage.

Regional failover / disaster recovery

Only the incident commander and designated SQL service owner may authorize a forced failover. Forced failover can lose committed transactions not replicated to the secondary. Practice planned failover in a nonproduction environment; never test forced failover in production without explicit

approval and a disaster declaration.

1. Confirm outage scope using Azure Service Health, primary/secondary health, app probes, replica synchronization/lag, affected databases, and recovery objectives. Check the configured failover group and its policy.
2. Decide whether the event meets the approved failover criteria. Record decision maker, expected RPO/RTO/data loss, application impact, and communication plan.
3. Confirm secondary server/pool capacity and database mapping, DNS/network/private endpoint resolution, identity, and dependent application services.
4. Execute the approved failover command/procedure. Prefer planned failover when the primary is healthy and data loss must be minimized; forced failover is a disaster action.
5. Validate writes/reads using the stable listener, app health, data consistency, downstream jobs, alerting, and dashboard region context.
6. Preserve evidence; communicate status; plan reverse failback only after the original region is healthy, replication is caught up, and a separate change is approved.
7. Run a post-incident review and update DR tests, architecture, automation, contacts, and recovery objectives.

Failover groups provide listener endpoints and can be used with selected databases, but regional recovery must include the rest of the application and its dependencies. Confirm current platform behavior, supported configuration, replication lag, data-loss risk, and failover policy in Microsoft documentation before the change.

7. Escalation and incident notes

Severity	Example	Response
Sev 1 / Critical	Broad production outage, confirmed availability loss, or imminent data-loss risk	Page primary on-call; open incident bridge; engage app, database, network, and Azure support owners; incident commander controls recovery/failover.
Sev 2 / High	SLO breach, sustained pool saturation with customer impact, rapidly approaching storage limit	Page/notify DBA and app owner; assign mitigation and frequent status updates.
Sev 3 / Warning	Elevated utilization, isolated workload regression, diagnostics gap without customer impact	Ticket to resource owner with trend, due date, and capacity/security action.
Informational	Planned maintenance, completed change, low-risk trend	Record and review in routine operations.

For all incidents record UTC timestamps, alert/rule/resource IDs, affected databases and applications, metrics/queries, user-visible impact, activity logs, actions and approvals, data-loss assessment, mitigation/rollback, communications, and follow-up owner/date. Never include secrets or sensitive query parameters in tickets.

8. Acceptance and handover checklist

- [ ] All SQL resources are inventoried, tagged, classified, and have named application/DBA owners.
- [ ] Pool placement, per-database limits, and capacity are supported by representative workload measurements and a cost review.
- [ ] RBAC, identity, private networking/DNS, policy, auditing, and security controls passed validation.
- [ ] Diagnostics are enabled for every pool and every database; workspace freshness and retention are verified.
- [ ] Metric/log alerts have tuned thresholds, severity, ownership, and tested action-group delivery.
- [ ] Fleet Workbook supports subscription/environment/server/pool/database drilldown and includes operational links and metric caveats.
- [ ] CI/IaC changes are reviewed, repeatable, least-privilege, and have nonproduction/rollback evidence.
- [ ] Restore and regional recovery have documented run steps, application

dependencies, approved RPO/RTO, and a recent successful test.

- [ ] On-call roster, incident matrix, change process, cost report, and ownership handover are complete.

9. Microsoft documentation

Recheck these sources during implementation because Azure features, CLI flags, metric names, available diagnostic categories, and pricing can change.

- Manage multiple databases with elastic pools
- Manage elastic pools
- Monitor Azure SQL Database with metrics and alerts
- Elastic pool metric reference
- Configure export of Azure SQL metrics and resource logs
- Azure Monitor Workbooks overview
- Azure SQL Database failover groups
- Azure Monitor alerts overview